WEEK 3: Polynomial and Multiple Regression

LINEAR REGRESSION

Regression analysis may broadly be defined as the analysis of relationships among variables. This relationship is given as an equation that helps to predict the dependent variable Y through one or more independent variables. In regression analysis, the variable whose values vary with the variations in the values of the other variable(s) is called the **dependent variable** or **response variable**. The other variables which are independent in nature and influence the response variable are called **independent variables**, **predictor variables** or **regressor variables**.

Example: Suppose a statistician employed by a cold drink bottler is analysing the product delivery and service operation for vending machines. He would like to find how the delivery time taken by the delivery man to load and service a machine is related to the volume of delivery cases. The statistician visits 50 randomly chosen retailer shops having vending machines and observes the delivery time (in minutes) and the volume of delivery cases for each shop. He plots those 50 observations on a graph, which shows that an approximate linear relationship exists between the delivery time and delivery volume. If Y represents the delivery time and X, the delivery volume, the equation of a straight line relating these two variables may be given as

Y = a + bX … (1)

where a is the intercept and b, the slope.

In such cases, we draw a straight line in the form of equation (1) so that the data points generally fall near the straight line. Now, suppose the points do not fall exactly on the straight line. Then we should modify equation (1) to minimise the difference between the observed value of Y and that given by the straight line (a + b X). This is known as **error**.

The error e, which is the difference between the observed value and the predicted value of the variable of interest Y, may be conveniently assumed as a statistical error. This error term accounts for the variability in Y that cannot be explained by the linear relationship between X and Y. It may arise due to the effects of other factors. Thus, a more plausible model for the variable of interest (Y) may be given as

Y = a + b X + e … (2)

where the intercept a and the slope b are unknown constants and e is a random error component. Equation (2) is called a **linear regression model**.

Fitting of regression line Let the given data of n pairs of observations on X and Y be as follows:

X: X_1 X_2 X_3 . … … Xi ….. … Xn

Y: Y_1 Y_2 Y_3 . … … Yi …..… Yn

where Y is the dependent variable and X, the independent variable.

Suppose, we wish to fit the following simple regression equation to the data: Y = a + bX

where a is the intercept and b is the slope of the equation.

For fitting equation to the data on (X, Y), we follow the steps given below:

Step 1: We draw a scatter diagram by plotting the (X, Y) points given in data.

Step 2: We construct a table as given below and take the sum of the values of X_i, Y_i, X_iY_i, and . We write the values of ΣX, ΣY, ΣXY and ΣX_2 in the last row.

Step 3: We express of aˆ given in equation (1) as follows:

$$\hat{a} = \bar{Y} - b\bar{X} = \frac{1}{n}[\sum Y - b\sum X]$$

$$\hat{b} = \frac{n\sum XY - \sum X\sum Y}{n\sum X^2 - (\sum X)^2}$$

Where

substitute above values in the regression equation and get

POLYNOMIAL REGRESSION

Some engineering data is poorly represented by a straight line. For these cases, a curve would be better suited to it these data. The method is to it polynomials to the data using polynomial regression.

Polynomial Regression is a form of regression analysis in which the relationship between the independent variables and dependent variables are modeled in the nth degree polynomial. Polynomial Regression models are usually fit with the method of least squares. The **least square method** minimizes the variance of the coefficients, under the Gauss Markov Theorem. Polynomial Regression is a special case of Linear Regression where we fit the polynomial equation on the data with a curvilinear relationship between the dependent and independent variables. A Quadratic Equation is a Polynomial Equation of 2nd Degree. However, this degree can increase to nth values.
The least-squares procedure can be readily extended to it the data to a higher-order polynomial. For example, suppose that we it a second-order polynomial or quadratic:

$$y = a_0 + a_1 x + a_2 x^2 + e$$

Where x-independent variable, y-dependent variable, a0,a1 and a2 are coefficients, and e –error term.
For this case the sum of the squares of the residuals is

$$S_r = \sum_{i=1}^{n} (y_i - a_0 - a_1 x_i - a_2 x_i^2)^2$$

we take the derivative with respect to each of the unknown coefficients of the polynomial, as in

$$\frac{\partial S_r}{\partial a_0} = -2 \sum (y_i - a_0 - a_1 x_i - a_2 x_i^2)$$

$$\frac{\partial S_r}{\partial a_1} = -2 \sum x_i (y_i - a_0 - a_1 x_i - a_2 x_i^2)$$

$$\frac{\partial S_r}{\partial a_2} = -2 \sum x_i^2 (y_i - a_0 - a_1 x_i - a_2 x_i^2)$$

These equations can be set equal to zero and rearranged to develop the following set of normal equations:

$$(n)a_0 + (\sum x_i)a_1 + (\sum x_i^2)a_2 = \sum y_i$$
$$(\sum x_i)a_0 + (\sum x_i^2)a_1 + (\sum x_i^3)a_2 = \sum x_i y_i$$
$$(\sum x_i^2)a_0 + (\sum x_i^3)a_1 + (\sum x_i^4)a_2 = \sum x_i^2 y_i$$

where all summations are from i = 1 through n. Note that the above three equations are linear and have three unknowns: a0 , a1 , and a2 . The coefficients of the unknowns can be calculated directly from the observed data. For this case, we see that the problem of determining a least-squares second-order polynomial is equivalent to solving a system of three simultaneous linear equations.
These equations can be rewritten in matrix form as follows:

$$(n)a_0 + (\sum x_i)a_1 + (\sum x_i^2)a_2 = \sum y_i$$
$$(\sum x_i)a_0 + (\sum x_i^2)a_1 + (\sum x_i^3)a_2 = \sum x_i y_i$$
$$(\sum x_i^2)a_0 + (\sum x_i^3)a_1 + (\sum x_i^4)a_2 = \sum x_i^2 y_i$$

Where:
- n is the number of data points.
- *xi* and *yi* are the values of the independent and dependent variables for the *i*-th data point.

The most common way to solve a system of linear equations is by using matrix algebra and methods like Gaussian elimination or matrix inversion. Here are the general steps to solve a system of linear equations:
Step 1: Write Down the System of Equations
Write the system of linear equations in standard form, where each equation has the form:

Where:

$$a_1x_1 + a_2x_2 + \ldots + a_nx_n = b$$

- $a1,a2,\ldots,an$ are the coefficients of the variables $x1,x2,\ldots,xn$.
- b is the constant on the right-hand side of the equation.

You should have as many equations as there are variables.

Step 2: Create a Coefficient Matrix

Construct a coefficient matrix A by extracting the coefficients of the variables from the left side of each equation. This matrix will have dimensions $n \times n$ if you have n variables.

Step 3: Create a Right-Hand Side Vector

Create a right-hand side vector B by extracting the constants from the right side of each equation. This vector will have dimensions $n \times 1$.

Step 4: Solve the System

There are several methods to solve the system:

There are several methods to solve the system:

- **Matrix Inversion**: If A is invertible (non-singular), you can solve the system using the formula $X=A^{-1}B$, where X is the vector of solutions.
- **Gaussian Elimination**: Use row operations to transform the augmented matrix $[A|B]$ into row-echelon or reduced row-echelon form. Then, back-substitute to find the values of the variables.
- **Matrix Factorization**: Factorize A into LU or QR form and use the factorization to solve the system more efficiently.
- **Numerical Methods**: For large or ill-conditioned systems, numerical methods like the Gauss-Seidel method or the Conjugate Gradient method are used.

Here's a simple example using matrix inversion in Python:

```
import numpy as np
# Define the coefficient matrix A and the right-hand side vector B
A = np.array([[2, 1], [1, 3]])
B = np.array([4, 7])
# Solve for the variables X using matrix inversion
X = np.linalg.inv(A).dot(B)
print("Solution:")
print(X)
```

The above code solves the system $2x_1+x_2=4$ and $x_1+3x_2=7$ and prints the values of x_1 and x_2 as the solution.

Solution:

[1. 2.]

Polynomial regression is a type of regression analysis in which the relationship between the independent variable (X) and the dependent variable (Y) is modeled as an nth-degree polynomial. In Python, you can perform polynomial regression using libraries like NumPy and scikit-learn. Here's a basic example of polynomial regression using scikit-learn:

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
import matplotlib.pyplot as plt
# Generate sample data
np.random.seed(0)
X = 2 * np.random.rand(100, 1)
Y = 4 + 3 * X + np.random.randn(100, 1)
# Fit a polynomial regression model
degree = 2 # You can change the degree as needed
```

```
poly_features = PolynomialFeatures(degree=degree)
X_poly = poly_features.fit_transform(X)

model = LinearRegression()
model.fit(X_poly, Y)
# Make predictions
X_new = np.linspace(0, 2, 100).reshape(-1, 1)
X_new_poly = poly_features.transform(X_new)
Y_new = model.predict(X_new_poly)
# Plot the original data and the polynomial regression curve
plt.scatter(X, Y, label='Original Data')
plt.plot(X_new, Y_new, 'r-', label='Polynomial Regression')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.show()
# The coefficients of the multivariate polynomial regression model
coefficients = model.coef_
intercept = model.intercept_
print("Coefficients:")
print(coefficients)
print("Intercept:")
print(intercept)
```

In this example:

1. We generate some sample data points with random noise.
2. We use PolynomialFeatures from scikit-learn to transform our input features X into polynomial features up to a specified degree.
3. We then fit a linear regression model to the polynomial features.
4. Finally, we make predictions using the model and plot the original data along with the polynomial regression curve.

You can adjust the degree variable to control the degree of the polynomial you want to fit to your data. Higher-degree polynomials can capture more complex relationships but may also lead to overfitting, so be cautious when choosing the degree.

```
model = LinearRegression()
model.fit(X_poly, Y)
# Make predictions
X_new = np.linspace(0, 2, 100).reshape(-1, 1)
X_new_poly = poly_features.transform(X_new)
Y_new = model.predict(X_new_poly)
# Plot the original data and the polynomial regression curve
plt.scatter(X, Y, label='Original Data')
plt.plot(X_new, Y_new, 'r-', label='Polynomial Regression')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.show()
# The coefficients of the multivariate polynomial regression model
coefficients = model.coef_
intercept = model.intercept_
print("Coefficients:")
print(coefficients)
print("Intercept:")
print(intercept)
```

In this example:

1. We generate some sample data points with random noise.
2. We use **PolynomialFeatures** from scikit-learn to transform our input features **X** into polynomial features up to a specified degree.
3. We then fit a linear regression model to the polynomial features.
4. Finally, we make predictions using the model and plot the original data along with the polynomial regression curve.

You can adjust the **degree** variable to control the degree of the polynomial you want to fit to your data. Higher-degree polynomials can capture more complex relationships but may also lead to overfitting, so be cautious when choosing the degree.

MULTIPLE LINEAR REGRESSION

Multiple linear regression is a method we can use to quantify the relationship between two or more predictor variables and a response variable.

The Regression Line: With one independent variable, we may write the regression equation as:

Where Y is an observed score on the dependent variable, *a* is the intercept, *b* is the slope, X is the observed score on the independent variable, and *e* is an error or residual.

We can extend this to any number of independent variables:

$$Y = a + b_1 X_1 + b_2 X_2 + \ldots + b_k X_k + e \quad (3.1)$$

Note that we have *k* independent variables and a slope for each. We still have one error and one intercept. Again we want to choose the estimates of *a* and *b* so as to minimize the sum of squared errors of prediction. The prediction equation is:

$$Y' = a + b_1 X_1 + b_2 X_2 + \ldots + b_k X_k \quad (3.2)$$

Finding the values of *b* (the slopes) is tricky for k>2 independent variables, and you really need matrix algebra to see the computations. It's simpler for k=2 IVs, which we will discuss here. But the basic ideas

are the same no matter how many independent variables you have. If you understand the meaning of the slopes with two independent variables, you will likely be good no matter how many you have.
For the one variable case, the calculation of *b* and *a* was:

$$b = \frac{\sum xy}{\sum x^2}$$

$$a = \overline{Y} - b\overline{X}$$

For the two variables in this case:

$$b_1 = \frac{(\sum x_2^2)(\sum x_1 y) - (\sum x_1 x_2)(\sum x_2 y)}{(\sum x_1^2)(\sum x_2^2) - (\sum x_1 x_2)^2}$$ and

$$b_2 = \frac{(\sum x_1^2)(\sum x_2 y) - (\sum x_1 x_2)(\sum x_1 y)}{(\sum x_1^2)(\sum x_2^2) - (\sum x_1 x_2)^2}$$

where
- $\Sigma x_1^2 = \Sigma X_1^2 - ((\Sigma X_1)^2 / n)$
- $\Sigma x_2^2 = \Sigma X_2^2 - ((\Sigma X_2)^2 / n)$
- $\Sigma x_1 y = \Sigma X_1 y - ((\Sigma X_1 \Sigma y) / n)$
- $\Sigma x_2 y = \Sigma X_2 y - ((\Sigma X_2 \Sigma y) / n)$
- $\Sigma x_1 x_2 = \Sigma X_1 X_2 - ((\Sigma X_1 \Sigma X_2) / n)$

At this point, you should notice that all the terms from the one variable case appear in the two variable case. In the two variable case, the other X variable also appears in the equation. For example, X_2 appears in the equation for b_1. Note that terms corresponding to the variance of both X variables occur in the slopes. Also note that a term corresponding to the covariance of X1 and X2 (sum of deviation cross-products) also appears in the formula for the slope.
The equation for *a* with two independent variables is:
This equation is a straightforward generalization of the case for one independent variable.

MLR MATRIX FORMULATION
Multiple linear regression is a generalized form of simple linear regression, in which the data contains multiple explanatory variables.
- For SLR, we observe pairs of variables.
- For MLR, we observe rows of variables.
- Each row (or pair) is called a case, a record, or a data point
- yi is the response (or dependent variable) of the ith observation
- There are p explanatory variables (or covariates, predictors, independent variables), and xik is the value of the explanatory variable xk of the ith case

Exercise 1

The data set of size n = 15 (Yield data) contains measurements of yield from an experiment done at five different temperature levels. The variables are y = yield and x = temperature in degrees Fahrenheit. The table below gives the data used for this analysis.

i	Temp.	Yield
1	50	3.3
2	50	2.8
3	50	2.9
4	70	2.3
5	70	2.6
6	70	2.1
7	80	2.5
8	80	2.9
9	80	2.4
10	90	3.0
11	90	3.1
12	90	2.8
13	100	3.3
14	100	3.5
15	100	3.0

a. Create a CSV file with sample data.
b. Write a Python function program to:
Find the fitted simple linear and polynomial regression equations for the given data.
c. Compare the coefficients obtained from manually intuitive and matrix formulation methods with your program.
d. Plot the scatterplot of the raw data and then another scatterplot with lines pertaining to a linear fit and a quadratic fit overlayed.
e. Compute the error, MSE, and RMSE.

Note: Do not use scikit-learn.

Exercise 2

When heart muscle is deprived of oxygen, the tissue dies and leads to a heart attack ("myocardial infarction"). Apparently, cooling the heart reduces the size of the heart attack. It is not known, however, whether cooling is only effective if it takes place before the blood, flow to the heart becomes restricted. Some researchers (Hale, et al, 1997) hypothesized that cooling the heart would be effective in reducing the size of the heart attack even if it takes place after the blood flow becomes restricted.
To investigate their hypothesis, the researchers conducted an experiment on 32 anesthetized rabbits that were subjected to a heart attack. The researchers established three experimental groups:

- *Rabbits whose hearts were cooled to 6° C within 5 minutes of the blocked artery ("**early cooling**")*
- *Rabbits whose hearts were cooled to 6° C within 25 minutes of the blocked artery ("**late cooling**")*
- *Rabbits whose hearts were not cooled at all ("**no cooling**")*

*At the end of the experiment, the researchers measured the size of the **infarcted** (i.e., damaged) **area** (in grams) in each of the 32 rabbits. But, as you can imagine, there is great variability in the size of hearts. The size of a rabbit's infarcted area may be large only because it has a larger heart. Therefore, in order*

*to adjust for differences in heart sizes, the researchers also measured the size of the **region at risk** for infarction (in grams) in each of the 32 rabbits.*

Infarc Area Group X2 X3

0.119 0.34 3 0 0
0.19 0.64 3 0 0
0.395 0.76 3 0 0
0.469 0.83 3 0 0
0.13 0.73 3 0 0
0.311 0.82 3 0 0
0.418 0.95 3 0 0
0.48 1.06 3 0 0
0.687 1.2 3 0 0
0.847 1.47 3 0 0
0.062 0.44 1 1 0
0.122 0.77 1 1 0
0.033 0.9 1 1 0
0.102 1.07 1 1 0
0.206 1.01 1 1 0
0.249 1.03 1 1 0
0.22 1.16 1 1 0
0.299 1.21 1 1 0
0.35 1.2 1 1 0
0.35 1.22 1 1 0
0.588 0.99 1 1 0
0.379 0.77 2 0 1
0.149 1.05 2 0 1
0.316 1.06 2 0 1
0.39 1.02 2 0 1
0.429 0.99 2 0 1
0.477 0.97 2 0 1
0.439 1.12 2 0 1
0.446 1.23 2 0 1
0.538 1.19 2 0 1
0.625 1.22 2 0 1
0.974 1.4 2 0 1

With their measurements in hand, the researchers' primary research question was: Does the mean size of the infarcted area differ among the three treatment groups — no cooling, early cooling, and late cooling — when controlling for the size of the region at risk for infarction?

A regression model that the researchers might use in answering their research question is:

$$y_i=(\beta_0+\beta_1 x_{i1}+\beta_2 x_{i2}+\beta_3 x_{i3})+\epsilon_i$$

where:

yi is the size of the infarcted area (in grams) of rabbit i
- *xi1 is the size of the region at risk (in grams) of rabbit i*
- *xi2 = 1 if early cooling of rabbit i, 0 if not*
- *xi3 = 1 if late cooling of rabbit i, 0 if not*

and the independent error terms εi follow a normal distribution with mean 0 and equal variance σ2.

Illustrates the need for being able to "translate" a research question into a statistical procedure. Often, the procedure involves four steps, namely:

- *formulating a multiple regression model*
- *determining how the model helps answer the research question*
- *checking the model*
- *and performing a hypothesis test (or calculating a confidence interval)*

a. Create a CSV file with sample data.
b. Write a Python function program to:
c. Find the fitted multiple linear regression equation for the given data.
d. Compare the coefficients obtained manually using intuitive and matrix formulation methods with your program.
e. Plot the data adorned with the estimated regression equation.
f. Compute the error, MSE, and RMSE.
Note: Do not use scikit-learn.

0.48 1.06 3 0 0
0.687 1.2 3 0 0
0.847 1.47 3 0 0
0.062 0.44 1 1 0
0.122 0.77 1 1 0
0.033 0.9 1 1 0
0.102 1.07 1 1 0
0.206 1.01 1 1 0
0.249 1.03 1 1 0
0.22 1.16 1 1 0
0.299 1.21 1 1 0
0.35 1.2 1 1 0
0.35 1.22 1 1 0
0.588 0.99 1 1 0
0.379 0.77 2 0 1
0.149 1.05 2 0 1
0.316 1.06 2 0 1
0.39 1.02 2 0 1
0.429 0.99 2 0 1
0.477 0.97 2 0 1
0.439 1.12 2 0 1
0.446 1.23 2 0 1
0.538 1.19 2 0 1
0.625 1.22 2 0 1
0.974 1.4 2 0 1

With their measurements in hand, the researchers' primary research question was: Does the mean size of the infarcted area differ among the three treatment groups — no cooling, early cooling, and late cooling — when controlling for the size of the region at risk for infarction?

A regression model that the researchers might use in answering their research question